

Build a Hyperlane USDC Warp Route

Self-hosted bridge: lock a real token on **Base** → mint it synthetic 1:1 on **your chain**. Worked example uses USDC → Lightchain, but any ERC-20 (or native coin) and any Hyperlane-registered chain works — see “Using a different token”.

Read first — what this is and is not. This is a **self-hosted** warp route you operate: your own validator signs, your own relayer delivers. It is **not** Circle’s official bridge and **not** a public one-click UI. Default security is **threshold-1** (a single validator) — that key can authorize mints, so it is a single point of trust. Plan for more validators before calling it trustless, and never let strangers rely on a route whose agents you don’t keep online.

0 Prerequisites

Funds	Ops wallet with ETH on Base (deploy + gas) and native gas on your destination chain (deploy + relay).
Host	Always-on machine with Docker (validator + relayer run 24/7).
Tooling	Node.js + Hyperlane CLI. Pin a known version, e.g. <code>@hyperlane-xyz/cli</code> . Agent image <code>ghcr.io/hyperlane-xyz/hyperlane-agent</code> .
Key	Deployer/signer keyfile, permissions <code>0600</code> , never in git . Load into the CLI via env var <code>HYP_KEY</code> — never echo or paste the value.

1 Register your destination chain with Hyperlane

Add your chain to the Hyperlane registry: a name, its **chainId / domain**, an **RPC**, and a deployed **Mailbox** + ISM factory. If your chain has no Hyperlane core yet, run `hyperlane core deploy` first to get a Mailbox. Agent chain metadata lives in `configs/agent-config.json`.

Example (Lightchain): name `lightchainai` · domain `9200` · RPC `https://rpc.mainnet.lightchain.ai` · Mailbox `0x142a9CEf00ACcAddB76283c49A1Bf37f20c1F00e`

2 Write the warp recipe `warp-route-deployment.yaml`

Base side = **collateral** wrapping Circle USDC. Your side = **synthetic** USDC that mints when a Base lock is delivered. Set `owner` to **YOUR_OPS_WALLET** (not a hardware/treasury wallet).

```
# Base collateral → <yourchain> synthetic. Owner = your anon deployer.
base:
  type: collateral
  token: "0x833589fCD6eDb6E08f4c7C32D4f71b54bdA02913" # Circle USDC on Base
  owner: "0xYOUR_OPS_WALLET"
  mailbox: "0xeA87ae93Fa0019a82A727bfd3eBd1cFCa8f64f1D" # Base mailbox
  decimals: 6
  name: "USD Coin"
  symbol: "USDC"
yourchain: # e.g. lightchainai
  type: synthetic
  owner: "0xYOUR_OPS_WALLET"
  mailbox: "0xYOUR_CHAIN_MAILBOX"
  decimals: 6
  name: "USD Coin"
  symbol: "USDC"
```

Using a different token? The route is token-agnostic. On the **origin** side set `token` to that ERC-20’s contract, and match its real `decimals` / `symbol` (e.g. 18 for most ERC-20s, 6 for USDC/

USDT). The **synthetic** side mirrors the same decimals/symbol. To bridge a chain's **native coin** instead of an ERC-20, set the origin `type: native` (no `token` field). One warp route = one token; deploy a separate route per coin.

3 Deploy the route

```
export PATH="$HOME/.npm-global/bin:$PATH"
# Load HYP_KEY from your local keyfile - do NOT print it
cd ~/.config/<your-deploy-dir>/warp-workdir
hyperlane warp deploy -y -w USDC/base-yourchain --verbosity info
unset HYP_KEY
```

Output (yours will differ): a `HypERC20Collateral` on Base (locks USDC) and a **synthetic HypERC20** on your chain (mints USDC). The CLI enrolls the two routers to each other and writes artifacts to `~/.hyperlane/`.

If the Base RPC flakes during deploy, edit `~/.hyperlane/chains/base/metadata.yaml` to list several RPCs (`mainnet.base.org`, `base.llamarpc.com`, `1rpc.io/base` ...) and re-run.

4 Choose your ISM (security module)

Point the synthetic token at an ISM that decides which messages are valid. Simplest bootstrap is a **TrustedRelayerIsM** (you relay). For self-verifying delivery, move to a **MessageIdMultisigIsM** with your validator announced on Base:

```
hyperlane ism deploy --chain yourchain -i ism-config.yaml -y --verbosity info
```

A live example runs **MessageIdMultisigIsM, threshold 1**, validator set = your one Base-announced validator. **Threshold 1 = single trust point** — add validators to harden.

5 Run validator + relayer (Docker, always-on)

```
cd ~/.config/<your-deploy-dir>
CONFIG_FILES="$(pwd)/configs/agent-config.json"
SIGDIR="$(pwd)/tmp/validator-signatures"
# KEY = load signer into a shell var from your keyfile; never echo

docker run -d --name hyp-validator-base --restart unless-stopped \
  --mount type=bind,source="$CONFIG_FILES",target=/config/agent-config.json,readonly \
  --mount type=bind,source="$(pwd)/db_validator",target=/hyperlane_db \
  --mount type=bind,source="$SIGDIR",target=/tmp/validator-signatures \
  -e CONFIG_FILES=/config/agent-config.json \
  ghcr.io/hyperlane-xyz/hyperlane-agent ./validator \
  --db /hyperlane_db --originChainName base \
  --checkpointSyncer.type localStorage \
  --checkpointSyncer.path /tmp/validator-signatures \
  --validator.key "$KEY"

docker run -d --name hyp-relayer --restart unless-stopped \
  --mount type=bind,source="$CONFIG_FILES",target=/config/agent-config.json,readonly \
  --mount type=bind,source="$(pwd)/db_relayer",target=/hyperlane_db \
  --mount type=bind,source="$SIGDIR",target=/tmp/validator-signatures,readonly \
  -e CONFIG_FILES=/config/agent-config.json \
  ghcr.io/hyperlane-xyz/hyperlane-agent ./relayer \
  --db /hyperlane_db --relayChains base,yourchain \
  --allowLocalCheckpointSyncers true --defaultSigner.key "$KEY"
```

Key hygiene: passing `--validator.key` on the command line leaves the hex visible via `docker inspect`. Prefer env/file injection with tight permissions. Keep the relayer wallet funded on your chain for gas.

6 Test send — \$1 end-to-end before you trust it

```
export PATH="$HOME/.npm-global/bin:$PATH"
# HYP_KEY loaded from keyfile – do not print
cd ~/.config/<your-deploy-dir>/warp-workdir
hyperlane warp send -y -w USDC/base-yourchain \
  --origin base --destination yourchain \
  --amount 1000000 --relay --verbosity info # 1000000 = 1.0 USDC (6 decimals)
unset HYP_KEY
```

A healthy 1 USDC send should show as processed on `explorer.hyperlane.xyz` once your relayer (or `--relay`) delivers it — often a couple of minutes. Anyone bridging afterward needs Base USDC + Base gas **and** your delivery online (Docker relayer running, or `--relay` on the send).

7 Consume the bridged USDC in your app (optional)

A minimal payment flow: user holds synthetic USDC on your chain → sends the required amount to your receiver wallet → your backend verifies by watching the token's ERC-20 `Transfer` event into that receiver (**not** a native-coin balance check). This is the pattern OrcaVault uses to accept “2 native or 1 bridged USDC” for ERC-8004 agent registration.

Reference addresses

Role	Address	Reuse?
Circle USDC (Base)	0x833589fCD6eDb6E08f4c7C32D4f71b54bdA02913	Universal — use as-is
Mailbox (Base)	0xeA87ae93Fa0019a82A727bfd3eBd1cFCa8f64f1D	Universal — use as-is
ValidatorAnnounce (Base)	0x182E8d7c5F1B06201b102123FC7dF0EaeB445a7B	Universal — use as-is
Mailbox (Lightchain ex.)	0x142a9CEf00ACcAddB76283c49A1Bf37f20c1F00e	Swap for your chain's mailbox
Collateral (Base)	generated by your deploy	Yours will differ
Synthetic USDC	generated by your deploy	Yours will differ
ISM / validator	your own values	Yours will differ
Owner / signer	your ops wallet	Yours — never a hardware/treasury key

Security checklist

1. Threshold-1 ISM means the validator key can authorize mints — protect it, add validators before calling it trustless.
2. Inject keys via env/files with tight permissions; avoid leaving hex in Docker `Cmd` (visible in `docker inspect`).
3. Never commit the deployer keyfile, agent keys, or API keys. Keep the deploy dir out of git.
4. Hyperlane CLI flags shift between versions — treat command shapes as guidance and confirm against current CLI docs before running.
5. Liquidity is whatever you bridge. Synthetic USDC on a new chain is working plumbing, not a market — don't market it as a liquid on-ramp.

Reference: Hyperlane docs (warp routes, ISMs, agents) at docs.hyperlane.xyz. Circle USDC + Hyperlane mailbox addresses are public infrastructure. All wallet, ISM, collateral, and synthetic addresses in your own route are generated during deploy and will differ from any example. — Built from a verified live route (Base → Lightchain).